CHAPTER 1

INTRODUCTION

1.1 About the Company

WhatsNext Vision Motors is an emerging force in the automotive industry, driven by a vision to reshape the future of mobility through technology, innovation, and customer-first strategies. With a deep commitment to efficiency and user satisfaction, the company continues to invest in cutting-edge digital solutions that enhance its business operations and customer relationships.

1.2 The Need for Transformation

In today's competitive market, delivering seamless and efficient customer experiences is no longer optional — it’s a necessity. WhatsNext Vision Motors identified critical challenges in its traditional customer interaction and order management processes, including dealer coordination, stock visibility, and operational delays. These gaps presented an opportunity for digital innovation.

1.3 Salesforce CRM Implementation

To address these challenges, the company launched a comprehensive Salesforce CRM project. The objective is clear: to simplify and automate customer-related workflows, increase visibility into stock levels, and ensure a faster, more personalized vehicle ordering experience. Salesforce was chosen for its scalability, robust feature set, and strong track record in CRM excellence.

1.4 Customer-Centric Ordering System

One of the central pillars of this project is an intelligent customer ordering system. This system is designed to automatically suggest the nearest dealership based on the customer's location, eliminating the need for manual dealer selection and reducing turnaround time. This not only improves the user experience but also optimizes logistical coordination between the company and its dealer network.

1.5 Real-Time Stock Validation

The system also addresses one of the most common issues in the automotive sector: stock availability. Customers can now only place orders for vehicles that are currently in stock. By preventing orders for unavailable products, the platform reduces customer frustration, minimizes cancellation rates, and ensures a more accurate and trustworthy ordering experience.

1.6 Automation & Efficiency

Automated workflows are another key feature of the implementation. These workflows handle everything from dealer assignment and order validation to status updates and email reminders for test drives. Additionally, batch Apex jobs ensure periodic checks on stock and order statuses. This level of automation significantly reduces the manual workload for staff, allowing them to focus on strategic tasks and customer engagement.

1.7 Outcomes and Vision

By integrating Salesforce into its operations, WhatsNext Vision Motors aims to establish a high-performance CRM system that supports both short-term efficiency and long-term growth. This initiative is expected to improve customer loyalty, enhance operational agility, and set a new standard for how technology can drive transformation in the automotive industry.

CHAPTER 2

MAJOR OBJECTIVE & SCOPE OF PROJECT

2.1 Primary Objective

The primary objective of the Salesforce CRM implementation at **WhatsNext Vision Motors** is to **transform the customer interaction and order management experience** through automation, accuracy, and real-time insights. The project aims to eliminate bottlenecks in the existing ordering process by:

- Enhancing the **convenience and personalization** of the customer experience.
- Providing **accurate, location-based dealer assignment** using automated workflows.
- Preventing **invalid or unfulfillable orders** by verifying stock availability in real-time.
- Keeping customers informed with **live order status updates**.
- Reducing dependency on manual tasks, thereby improving **operational efficiency**.

This transformation aligns with the company's long-term goal of delivering **smarter, faster, and more reliable automotive services** that align with modern digital expectations.

2.2 Scope of the Project

The Salesforce CRM implementation covers a wide range of business functionalities, spanning data management, automation, backend logic, and customer communication. The project scope includes:

2.2.1. Vehicle & Dealer Data Management

- Create and manage **custom Salesforce objects** for vehicles, dealers, stock inventory, customer orders, test drives, and service requests.
- Establish **relationships between objects** to link customers with orders, dealers, and test drives efficiently.

- Use standard and custom fields to maintain **structured and searchable records** for easy access and analytics.

2.2.2. Intelligent Order Workflow Automation

- **Auto-suggest the nearest dealership** based on customer address using geolocation logic or predefined mappings.
- Implement **record-triggered flows** and workflow rules that:
    - Prevent order submission if the vehicle is not available in stock.
    - Instantly assign the dealer based on the customer's pincode or city.
    - Trigger confirmation emails or SMS notifications upon order placement.

2.2.3. Business Logic via Apex Triggers

- Write **Apex triggers** to enforce critical business rules such as:
    - Rejecting orders for out-of-stock vehicles.
    - Automatically creating records for follow-ups or status checks.
- Utilize **trigger handlers** to ensure modularity, readability, and ease of maintenance.
- Handle exceptions and edge cases like:
    - Stock depletion during order processing.
    - Duplicate order requests.

2.2.4. Batch Jobs & Scheduled Apex

- Design and deploy **Batch Apex classes** to:
    - Periodically check stock levels.
    - Update bulk orders where stock status has changed.
- Implement **Scheduled Apex jobs** to:

    - Run stock validations at defined intervals (e.g., daily at midnight).
    - Send automated email alerts to admins when certain vehicle models fall below threshold stock levels.
    - Change the status of pending orders to "Confirmed" or "On Hold" based on updated inventory.

2.2.5. Test Drive & Service Management

- Automate scheduling and confirmation of **test drive requests**.
- Send **automated reminders** to both customer and dealer before the test drive time.
- Maintain logs for customer preferences and past interactions to offer personalized service.

2.2.6. Enhanced Customer Experience

- Ensure customers:
    - Cannot place invalid or unserviceable orders.
    - Receive **live status updates**: Pending, Confirmed, Dispatched, or Out of Stock.
    - Get test drive confirmations and dealer contact details via email.

2.2.7. Internal Efficiency Boost

- Reduce employee workload on repetitive tasks through:
    - Automation of order assignment, validation, and notification.
    - Centralized data visibility via Salesforce dashboards and reports.
- Improve collaboration between sales, service, and logistics teams.

CHAPTER 3

PROBLEM ANALYSIS AND REQUIREMENT SPECIFICATION

3.1 Problem Analysis

As WhatsNext Vision Motors scaled its operations, it encountered multiple business bottlenecks and technical limitations that hindered customer satisfaction and internal efficiency. These gaps formed the foundation for the Salesforce CRM implementation initiative.

3.1.1. Inefficient Dealer Assignment Workflow

Customers placing vehicle orders were not systematically connected to the nearest available dealer. The absence of automated dealer selection led to:

- Delays in order processing and fulfillment.
- Increased manual workload for sales coordinators.
- Mismatches between customer locations and dealer availability. This manual process also introduced inconsistencies, especially in high-demand regions or during promotional campaigns.

3.1.2. Inaccurate and Unverified Stock Management

The legacy system lacked integration with real-time inventory. As a result:

- Customers could place orders for vehicles that were no longer available in stock.
- Sales staff had to manually verify stock and communicate back, leading to delays.

- Customer dissatisfaction increased due to false expectations and post-order cancellations.

This created operational friction, reduced trust, and impacted brand credibility.

3.1.3. Lack of Real-Time Order Status Updates

The previous system did not have a dynamic order tracking system. Key issues included:

- No automatic update of order status based on stock changes or dealer assignment.
- Customers had to contact customer service multiple times for updates.
- Dealers were not consistently informed about new or pending orders. This lack of transparency led to poor customer experience and high support overhead.

3.1.4. Manual and Unreliable Test Drive Scheduling

Test drive bookings were handled through disconnected communication channels (calls, emails), resulting in:

- Missed test drives due to lack of timely reminders.
- Poor coordination between sales executives and customers.
- No centralized record of test drive requests or completion. This impacted sales conversions and made it harder to track customer interest.

3.1.5. Absence of Structured Data Models

The company did not have a proper data model in place to:

- Organize customer records, vehicle listings, service history, and dealer profiles.
- Define relationships between these entities for better reporting and decision-making. This led to data redundancy, poor visibility, and difficulties in performing analytics.

3.1.6. High Manual Workload and Operational Overhead

Due to the lack of automation and centralized systems:

- Employees had to perform repetitive tasks manually (status updates, dealer coordination, stock checks).
- Time was wasted on activities that could be handled by scheduled or triggered processes.
- Human error risk was high, especially under peak load conditions. This limited scalability and reduced productivity across departments.

3.2 Requirement Specification

Based on the challenges identified, the following functional and non-functional requirements were defined for the Salesforce CRM project.

3.2.1. Functional Requirements

a. Centralized Data Management

- Custom Salesforce objects for Vehicles, Orders, Customers, Dealers, Test Drives, and Services.
- Defined relationships between objects (e.g., Vehicle ↔ Order, Customer ↔ Dealer).

b. Dealer Auto-Assignment Logic

- Automatically assign orders to the nearest dealer using location-based filters.
- Ensure fallback mechanisms if a dealer is unavailable in a specific region.

c. Stock Availability Validation

- Validate vehicle stock before order submission.
- Prevent creation of orders for out-of-stock vehicles.

- Allow stock-based order filtering and reporting.

d. Dynamic Order Status Updates

- Use scheduled processes and record-triggered flows to:
    - Update order status to Pending if stock is unavailable.
    - Set status to Confirmed if the vehicle is in stock and a dealer is assigned.
- Send email/SMS notifications on status changes.

e. Test Drive Scheduling Automation

- Let users request test drives via a portal or CRM interface.
- Assign requests to dealers based on location.
- Send automated email reminders to both customer and dealer before the scheduled time.

f. Batch Processing and Scheduling

- Use Batch Apex jobs to check stock and update bulk order statuses.
- Implement Scheduled Apex to:
    - Send daily inventory alerts.
    - Remind sales teams about pending follow-ups.

g. Role-Based Access Control

- Secure access to sensitive records using Salesforce Profiles and Permission Sets.
- Separate permissions for Sales Executives, Dealers, Admins, and Customers.

2. Non-Functional Requirements

a. Performance

- The system should support concurrent operations from multiple users.
- Batch processes must be optimized for large data volumes without hitting governor limits.

b. Scalability

- Design the system to support future expansion, such as new regions, additional vehicle categories, or marketing integrations.

c. Security

- Protect customer data and business-critical information using Salesforce's built-in security model.
- Enable field-level and record-level access restrictions.

d. Accessibility

- Ensure compatibility across major browsers: Chrome, Firefox, Safari, Edge.
- Use Lightning Components to deliver a responsive and user-friendly interface.

e. Maintainability

- Follow best practices for Apex trigger handlers, modular classes, and flow naming conventions.
- Ensure that code is well-commented and easy to extend or debug.

CHAPTER 4

HARDWARE/SOFTWARE PLATFORM ENVIORNMENT

4.1. Hardware / Software Platform Environment

To ensure smooth development, deployment, and operation of the Salesforce CRM project at WhatsNext Vision Motors, a stable and compatible hardware-software environment is essential. The platform environment is defined to support Salesforce Lightning Experience, Apex development, Flow automation, and real-time data processing.

4.1.1. Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 (8th Gen) / AMD Ryzen 3	Intel Core i5 or higher / AMD Ryzen 5
RAM	4 GB	8 GB or more
Storage	10 GB free disk space	20 GB or more free disk space
Display	1366 × 768 resolution	1920 × 1080 (Full HD)
Network	Stable broadband connection (min. 3 Mbps)	10+ Mbps internet connection
Browser Support	At least 2 modern browsers installed	Chrome + Firefox / Safari / Edge

4.1.2. Software Requirements

Component	Description

Operating System	Windows 10/11, macOS (latest), Linux (browser-based access), ChromeOS
Browser	Google Chrome (latest), Mozilla Firefox, Microsoft Edge, Safari (Mac only)
Development Platform	Salesforce Lightning Platform
Apex Development	Salesforce Developer Console, VS Code with Salesforce CLI
Automation Tools	Salesforce Flow, Process Builder, Workflow Rules
Database	Salesforce Cloud Database (Object-based storage)
Version Control (Optional)	Git (for code backup and collaboration)
Email	Integration with Salesforce Email Services / Gmail / Outlook

4.1.3. Platform & Services

Service Type	Tool / Platform
CRM Platform	Salesforce Sales Cloud, Lightning Experience
Dev Tools	Salesforce CLI, Apex, Visualforce (if needed)
Automation	Flows, Process Builder, Scheduled Apex
Batch Jobs	Batch Apex, Queueable Apex
Notification System	Email Alerts, Platform Events
Security	Role-Based Access, Profiles, Permission Sets
Testing	Developer Console Tests, Debug Logs, Sandbox

4. Network & Security Configuration

- A **stable internet connection** is mandatory for all Salesforce development and deployment tasks.

- Ensure firewall settings allow access to the following Salesforce domains:
    - *.salesforce.com
    - *.force.com
    - *.visual.force.com
- **VPNs** or firewalls that block access to these domains should be disabled or configured to permit access.

CHAPTER 5

SNAPSHOTS OF INPUT OUTPUT & CODING

5.1. Basic UI of the Project

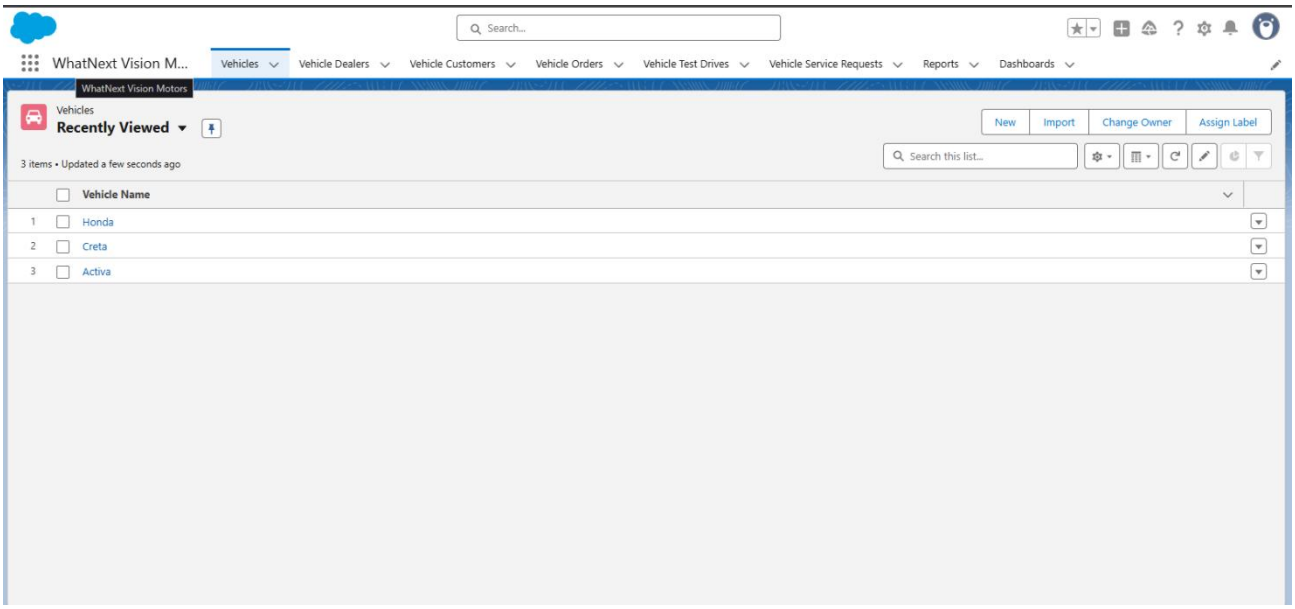

Fig. 5.1.1: Vehicles Module

Displays the "Vehicles" section, showing a list of recently viewed vehicles. The visible entries are: Honda, Creta, and Activa. The interface allows you to create a new entry, import data, change owner, or assign labels to vehicles.

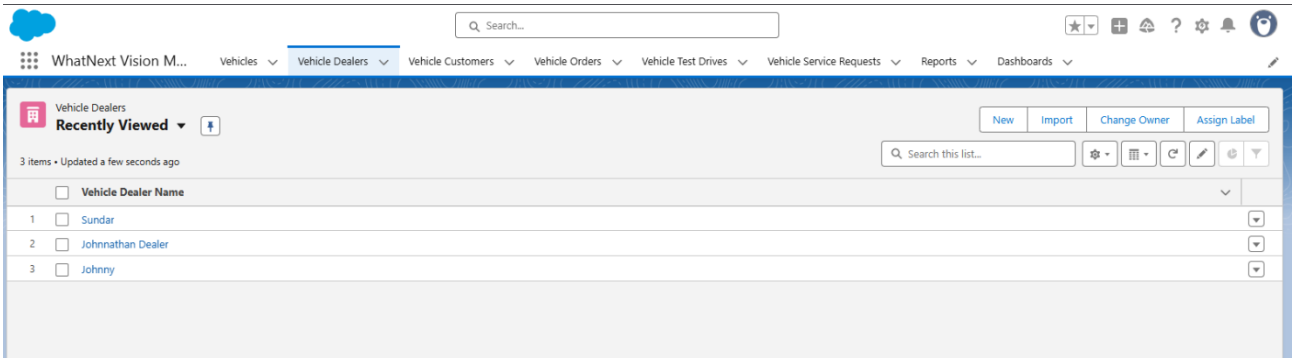

Fig. 5.1.2: Vehicle Dealers Module

Shows the "Vehicle Dealers" module, listing three dealers: Sundar, Johnnathan Dealer, and Johnny. The page offers similar functionality to add, import, change owner, or assign labels to vehicle dealers.

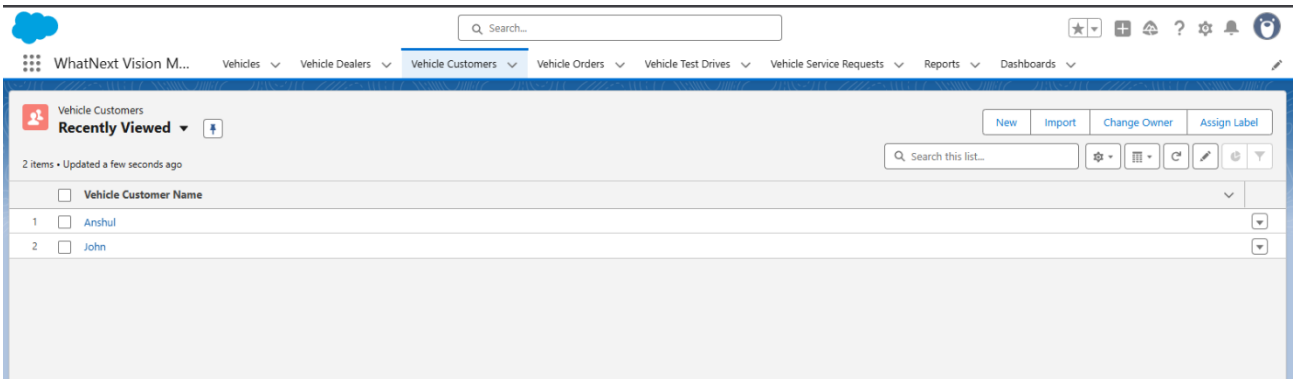

Fig. 5.1..3: Vehicle Customers Module

Presents the "Vehicle Customers" screen with two customer records: Anshul and John. The options for managing these customer records match those in the other modules, supporting additions, import, and ownership changes.

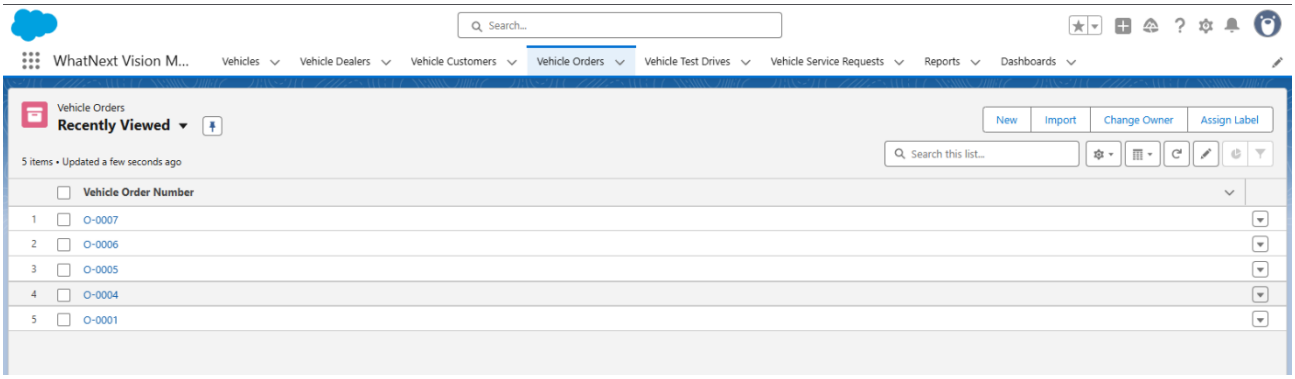

Fig. 5.1.4: Vehicle Orders Module

Shows the "Vehicle Orders" module, listing five vehicle orders by order number (O-0007, O-0006, O-0005, O-0004, O-0001). Users can create new orders, import existing ones, change the owner, or assign a label to each order4

5.2. Record-Triggered Flows

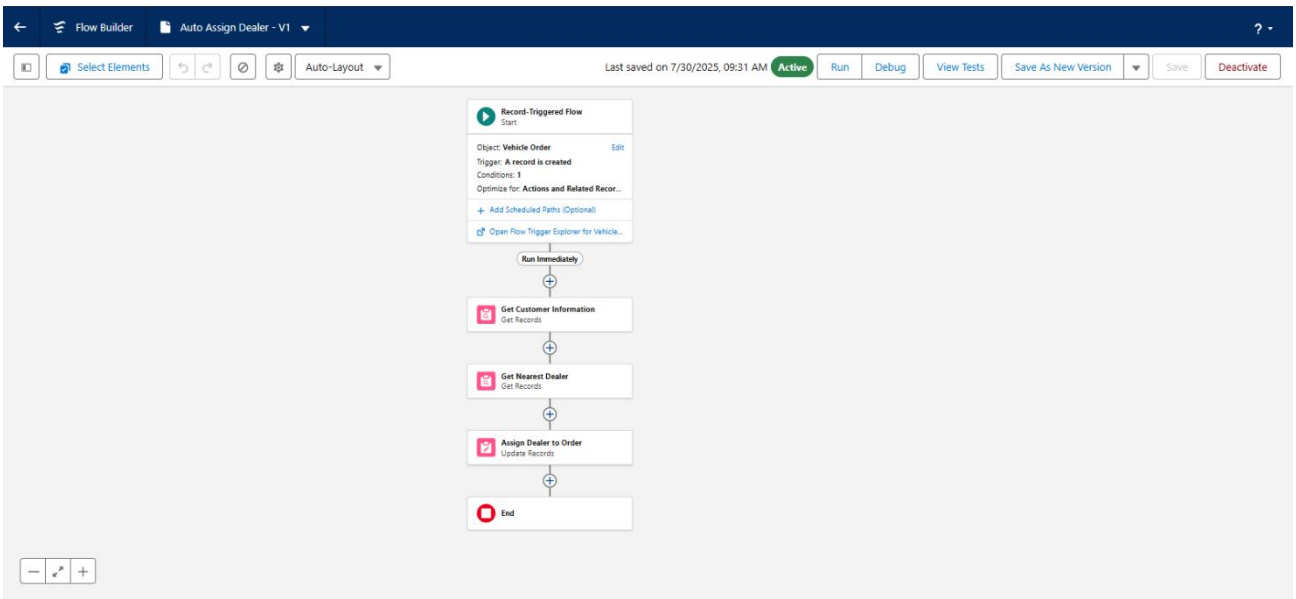

Fig. 5.2.1: Auto Assign Dealer Flow

This record-triggered flow is activated whenever a new Vehicle Order is created in Salesforce CRM. It immediately begins by retrieving detailed information about the customer associated with the order, ensuring that relevant customer data is available for processing. After fetching customer details, the flow identifies the nearest available dealer, likely using criteria such as geographic proximity or dealer availability, to optimize the assignment process. The selected dealer is then automatically linked to the vehicle order, updating the order record with the appropriate dealer details. This automation eliminates manual intervention, streamlines dealership assignments, and reduces the risk of human error. By automating dealer allocation, the business achieves faster order processing and ensures efficient management of vehicle orders from initiation to fulfilment.

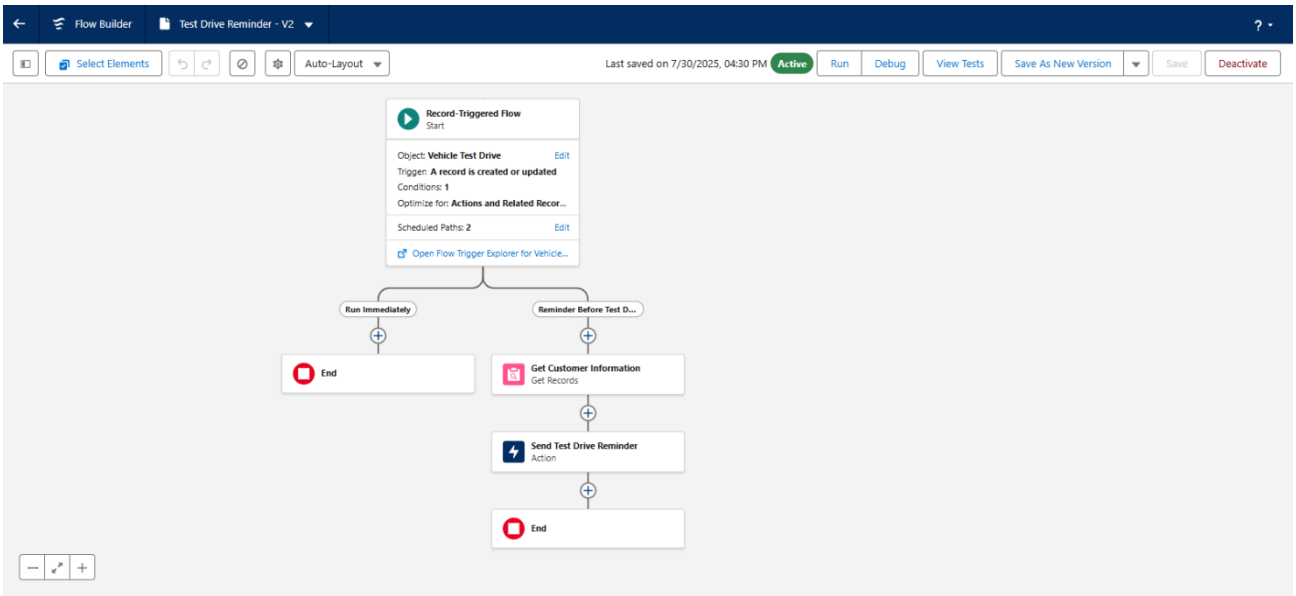

Fig. 5.2.2: Test Drive Reminder Flow

This flow is triggered when a Vehicle Test Drive record is either created or updated. It supports multiple paths, including one that executes actions right away and another that schedules actions to occur before the test drive event. The scheduled reminder path is particularly crucial: it starts by obtaining comprehensive customer information to personalize the communication. It then sends an automated reminder about the upcoming test drive, which could be through email or another notification method. This ensures customers are informed and reminded in a timely manner, which helps reduce the number of missed test drive appointments. The flow is designed to improve customer engagement, streamline test drive management, and offer an enhanced, proactive customer experience by leveraging automated communications.

5.3. Coding Involved in the Project (Apex)

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders);
        }

        if (isAfter && (isInsert || isUpdate)) {
            updateStockOnOrderPlacement(newOrders);
        }
    }

    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
                [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
            );

            for (Vehicle_Order__c order : orders) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle != null && vehicle.Stock_Quantity__c <= 0) {
                    order.addError('This vehicle is out of stock. Order cannot be placed.');
                }
            }
        }
    }
```

Fig. 5.3.1: VehicleOrderTriggerHandler

This Apex class provides logic to manage Salesforce triggers for the Vehicle Order object. Specifically, it prevents placing orders for vehicles that are out of stock, and it updates vehicle stock automatically when an order's status is changed to 'Confirmed'.

The handleTrigger method determines, based on the context (before/after, insert/update), whether to check for out-of-stock orders or to update stock quantities. The class includes helper functions: preventOrderIfOutOfStock (which blocks an order if no stock is available) and updateStockOnOrderPlacement (which decrements vehicle stock when an order is confirmed). This ensures data consistency and enforces business rules for vehicle inventory and order management.

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c FROM Vehicle_Order__c WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
                [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
            );

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

            for (Vehicle_Order__c order : orderList) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle != null && vehicle.Stock_Quantity__c > 0) {
                    order.Status__c = 'Confirmed';
                    vehicle.Stock_Quantity__c -= 1;
                    ordersToUpdate.add(order);
                    vehiclesToUpdate.add(vehicle);
                }
            }
```

Fig. 5.3.2: VehicleOrderBatch

This file defines an Apex batch class (VehicleOrderBatch) that implements the Database.Batchable interface. Its purpose is to automate the processing of pending vehicle orders in bulk. The batch job retrieves all vehicle orders with a 'Pending' status, checks the stock level for each related vehicle, and, if the order is 'Confirmed' and stock is available, it decrements the stock count accordingly. This batch processing is crucial for handling large datasets efficiently, ensuring that order and inventory records stay synchronized and reducing manual intervention.

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 = batch size
    }
}
```

Fig. 5.3.3: VehicleOrderBatchScheduler

This is an Apex class implementing the Schedulable interface, allowing the batch job (VehicleOrderBatch) to be scheduled and executed automatically at regular intervals (such as nightly or hourly runs). The execute method starts the batch job with a specified batch size (in this case, 50 records per batch), enabling scalable and automated background processing of vehicle orders and stock updates. This is important for continuous management of inventory and fulfillment without manual intervention, especially in high-volume environments.

```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore, Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
}
```

Fig. 5.3.4: VehicleOrderTrigger

This screenshot shows the code for an Apex trigger called VehicleOrderTrigger on the custom object Vehicle_Order__c. The trigger is set to execute before and after insert and update operations. Inside the trigger body, it calls a handler method from the VehicleOrderTriggerHandler class, passing important context variables such as the new and old data records, and Boolean flags for the trigger context (before/after, insert/update). This approach follows Salesforce best practices by delegating complex logic to a handler class, keeping the trigger streamlined, readable, and easier to maintain. It enables the business logic to be centralized and reusable across different trigger events.

CHAPTER 6

PROJECT LIMITATIONS AND FUTURE SCOPE

6.1. Project Limitations

While the Salesforce CRM implementation at WhatsNext Vision Motors marks a significant leap in customer experience and operational efficiency, there are some **inherent limitations** in the current version of the project that need to be acknowledged:

6.1.1. Limited Integration with External Systems

- The current implementation is focused primarily within the Salesforce ecosystem.
- Integration with external systems like ERP, payment gateways, SMS APIs, and logistics platforms is not yet established.
- As a result, real-time synchronization with finance, warehouse, or delivery tracking systems is currently unavailable.

6.1.2. Static Dealer Assignment Logic

- Dealer assignment is based on predefined location filters (e.g., city or pincode).
- The logic does not currently account for dealer workload, capacity, or business priority, which could lead to uneven distribution of orders.

6.1.3. No Real-Time Stock Sync with Physical Warehouses

- Stock data is managed manually within Salesforce or updated via batch jobs.
- There is no direct integration with warehouse management systems or inventory APIs, which limits real-time accuracy.

6.1.4. Basic Test Drive Management

- Test drive scheduling and reminders are functional but basic.
- Features like rescheduling, customer feedback, route optimization, and lead scoring based on test drive completion are not implemented yet.

6.1.5. Limited Reporting and Dashboards

- While standard Salesforce reports are available, advanced analytics, custom dashboards, and visual KPIs are minimal.
- Real-time data-driven decision-making tools for managers are yet to be built.

6. Single-Region Focus

- The system is currently optimized for a single-region operation.
- Scaling to multiple states or international territories may require additional localization, dealer zoning logic, and data compliance measures (e.g., GDPR).

6.2. Future Scope

The current project lays a strong foundation for digital transformation. However, several advanced features and capabilities can be introduced in future phases to enhance scalability, intelligence, and customer satisfaction.

6.2.1. Third-Party Integrations

- **ERP Integration** for financial tracking, invoicing, and cost management.
- **Logistics Integration** for vehicle dispatch, delivery tracking, and notifications.
- **Payment Gateway Integration** for secure online booking and partial payments.

6.2.2. AI-Powered Recommendations

- Use **Einstein AI** or other ML models to suggest vehicles based on customer history, preferences, or behavior.
- AI-based dealer assignment based on proximity, availability, and historical performance.

6.2.3. Real-Time Inventory Sync

- Integrate with **real-time inventory APIs** to maintain up-to-date stock visibility.
- Trigger automated reorder alerts when inventory drops below a threshold.

6.2.4. Advanced Test Drive Management

- Add features such as:
    - Live GPS tracking of test drive vehicles.
    - Instant rescheduling or cancellation options.
    - Feedback forms to collect customer experience post-test drive.

6.2.5. Enhanced Customer Portal

- Build a customer-facing portal or mobile app where users can:
    - Track their orders and test drive status.
    - View service history and book appointments.
    - Receive promotions, news, and support.

6.2.6. Scalability for Multi-Location Operations

- Enable support for **multi-regional operations** with dynamic zoning, dealer tiering, and language/localization support.
- Add compliance tracking for different legal and regulatory environments.

6.2.7. Advanced Analytics & Reporting

- Use Salesforce Einstein Analytics or integrate with tools like Tableau or Power BI.
- Develop **role-specific dashboards** for sales teams, service managers, and executives.